Web scraping and Social Media Scraping

Scraping static pages

Maciej Świtała, PhD
Ewa Weychert, PhD

Aim of today's class

- **Navigation in website structure (recapitulation)**
  - **Hierarchical structures:** XML and HTML organize data into a Document Object Model (DOM) tree where elements are nested as parent-child nodes.
  - **Navigation Tools:**
    - *CSS Selectors:* Target elements based on styling attributes.
    - *XPath:* Navigates the XML/HTML path logic (e.g., `//div[@class='price']`).
    - *Regex:* Used for pattern matching within raw text strings.
- **Scraping static websites**
  - **Static vs. Dynamic:** Static sites serve pre-rendered HTML directly from the server; dynamic sites require JavaScript execution to load content.
  - **Scaling:** Increasing throughput via asynchronous requests
  - **Asynchronous (Non-blocking):**
    - Instead of "Wait → Get Data → Wait," it's "Request → Request → Request". Don't sit idle. If one website is slow, move to the next one immediately.

throughput refers to the volume of data or the number of requests successfully processed within a specific time frame

Presentation Outline

How we find data?

- **The "Map" (Website Structure)**
  - HTML and XML are like **Family Trees**.
  - Data sits in "Nodes" (the branches or leaves of the tree).
  - To get the price of a product, you have to follow the right branch.
- **The "GPS" (Tools to move around)**
  - **CSS Selectors:** Finding data by "Style" (e.g., "Find the red text").
  - **XPath:** Finding data by "Address" (e.g., "Go to the 3rd floor, 2nd door on the left").
  - **Regex:** Finding data by "Pattern" (e.g., "Find anything that looks like a phone number").
- **How we do it:**
  - **Manual:** We look at the code ourselves (Right-click → Inspect).
  - **Plugins:** Tools like "" that highlight the path for us.

Example of an XML Structure

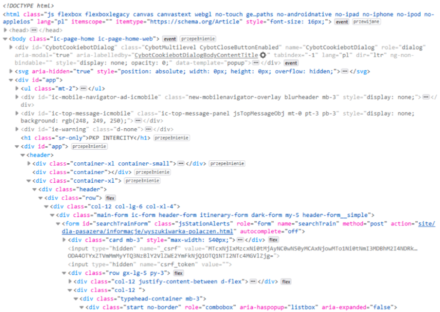
```xml
▼<bookstore>
  ▼<bookshelf id="left">
    ▼<book>
        <title lang="pl" cover="soft">Ubik</title>
        <price>45.99</price>
        <pages>304</pages>
        <author>Philip K. Dick</author>
      </book>
    ▼<book>
        <title lang="pl" cover="hard">Mitologia nordycka</title>
        <price>23.20</price>
        <pages>208</pages>
        <author>Neil Gaiman</author>
      </book>
    ▼<book>
        <title lang="en" cover="soft">Ender's Game</title>
        <price>38.99</price>
        <pages>384</pages>
        <author>Orson Scott Card</author>
      </book>
    </bookshelf>
```

A "Parent" tag (<bookstore>) holding "Children" (<book>)

Simple Breakdown:

- **Tags = Labels**
  `<price>` tells the computer "this number is money."
- **Attributes = Details**
  `id="left"` or `lang="en"` give extra info about the tag.
- **Hierarchy = Paths**
  To get the author, you follow the map:
  Bookshelf → *Book* → *Author*

Example of an HTML Structure



```
<!DOCTYPE html>
<html class="js flexbox flexboxlegacy canvas canvastext webgl no-touch ge…paths no-androidnative no-ipad no-iphone no-ipod no-appleios" lang="pl" itemscope="" itemtype="https://schema.org/Article" style="font-size: 16px;"> event przewijane
  <head> ··· </head>
  <body class="ic-page-home ic-page-home-web"> event przepełnienie
    <div id="CybotCookiebotDialog" class="CybotMultilevel CybotCloseButtonEnabled" name="CybotCookiebotDialog" role="dialog" aria-modal="true" aria-labelledby="CybotCookiebotDialogBodyContentTitle" tabindex="-1" lang="pl" dir="ltr" ng-non-bindable="" style="display: none; opacity: 0;" data-template="popup"> ··· </div> event
    <svg aria-hidden="true" style="position: absolute; width: 0px; height: 0px; overflow: hidden;"> ··· </svg>
    <div id="app">
      <ul class="mt-2"> ··· </ul>
      <div id="ic-mobile-navigator-ad-icmobile" class="new-mobilenavigator-overlay blurheader mb-3" style="display: none;"> ··· </div>
      <div id="ic-top-message-icmobile" class="ic-top-message-panel jsTopMessageObj mt-0 pt-3 pb-3" style="display: none; background: rgb(248, 249, 250);"> ··· </div>
      <div id="ie-warning" class="d-none"> ··· </div>
      <h1 class="sr-only">PKP INTERCITY</h1> przepełnienie
      <div id="app"> przepełnienie
        <header>
          <div class="container-xl container-small"> ··· </div> przepełnienie
          <div class="container"></div> przepełnienie
          <div class="container-xl"> przepełnienie
            <div class="header">
              <div class="row"> flex
                <div class="col-12 col-lg-6 col-xl-4">
                  <div class="main-form ic-form header-form itinerary-form dark-form my-5 header-form__simple">
                    <form id="searchTrainForm" class="jsStationAlerts" role="form" name="searchTrain" method="post" action="site/dla-pasazera/informacje/wyszukiwarka-polaczen.html" autocomplete="off">
                      <div class="card mb-3" style="max-width: 540px;"> ··· </div> flex
                      <input type="hidden" name="_csrf" value="MTcxNjIxMzcxNi0tMjAyNC0wNS0yMCAxNjowMTo1Ni0tNmI3MDBhM2I4NDRk…ODA4OTYxZTVmMmMyYTQ3NzBlY2VlZWE2YmFkNjQ1OTQ1NTI2NTc4MGVlZjg=">
                      <input type="hidden" name="csrf_token" value="">
                      <div class="row gx-lg-5 py-3"> flex
                        <div class="col-12 justify-content-between d-flex"> ··· </div> flex
                        <div class="col-12 ">
                          <div class="typehead-container mb-3">
                            <div class="start no-border" role="combobox" aria-haspopup="listbox" aria-expanded="false">
```

Source: intercity.pl/pl.

Real-world HTML from *intercity.pl*

Why is this so messy?

- **Visual Layers:** Unlike XML (which only holds data), HTML tells the browser how to *look*.
- **The "Div" Jungle:** Most boxes are just named `<div>`. We use **Classes** (like `class="row"`) to tell them apart.
- **Hidden Data:** Many tags hold info for the server (like `type="hidden"`), not for the user.

When to use XML vs. HTML in Scraping

XML

(The Data Specialist)

- **Purpose:** Designed to carry and store data.
- **Logic:** Very strict. Tags describe the content (e.g., `<price>`).
- **Scraping:** "Easy Mode." Clean and predictable.
- **Where:** APIs, RSS Feeds, Sitemaps.

HTML

(The Visual Specialist)

- **Purpose:** Designed to display data to humans.
- **Logic:** Messy. Tags describe the look (e.g., `<div>`, `<b>`).
- **Scraping:** "Hard Mode." You must filter out the "noise."
- **Where:** Every standard webpage.

Pro-Tip: Always check for an XML/API source first. It saves you from fighting with messy HTML layouts! **Project Note:** Since we aren't using APIs, you must learn to navigate the **HTML structure** to "hunt" for your data manually!

How to Find Data: Manual vs. Plugins

Manual Navigation
(Like using a Paper Map)

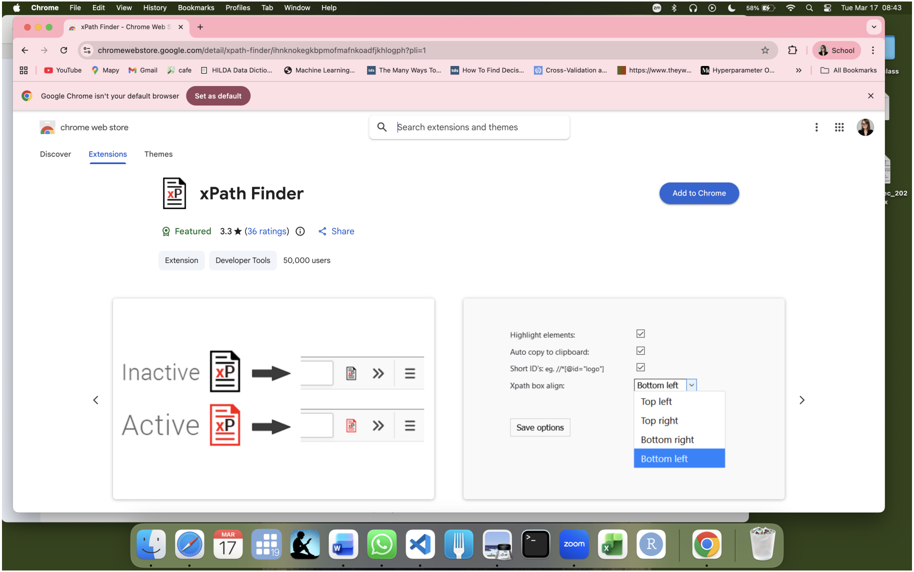
- **Action:** Right-click → "Inspect."
- **Process:** You read the messy HTML code line by line.
- **Problem:** It is **tedious** and **frustrating**. It's easy to get lost in the "Div Jungle."

Browser Plugins
(Like using Google Maps)

- **Tool:** ChroPath or SelectorGadget.
- **Action:** Just click on the item you want (Price, Title).
- **Benefit:** The plugin **instantly** gives you the "GPS Coordinates" (XPath/CSS).

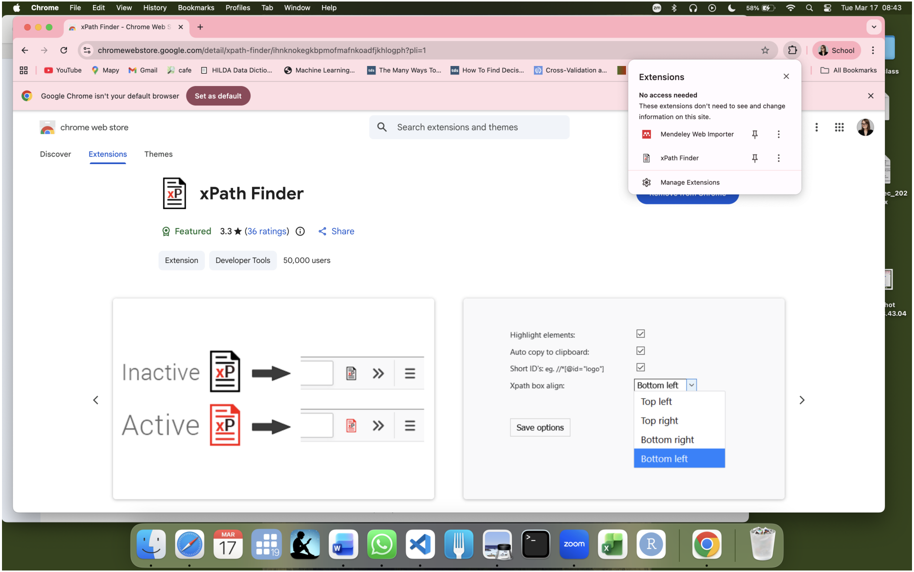
Student Tip: Don't be a hero. Use a plugin to find the path in 2 seconds instead of 20 minutes!

ChroPath plugin

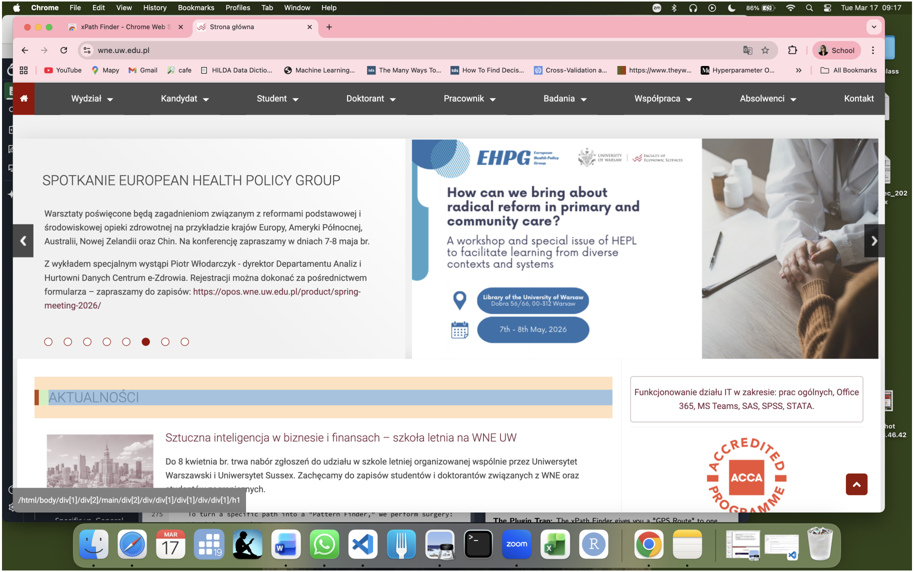


https://chromewebstore.google.com/detail/xpath-

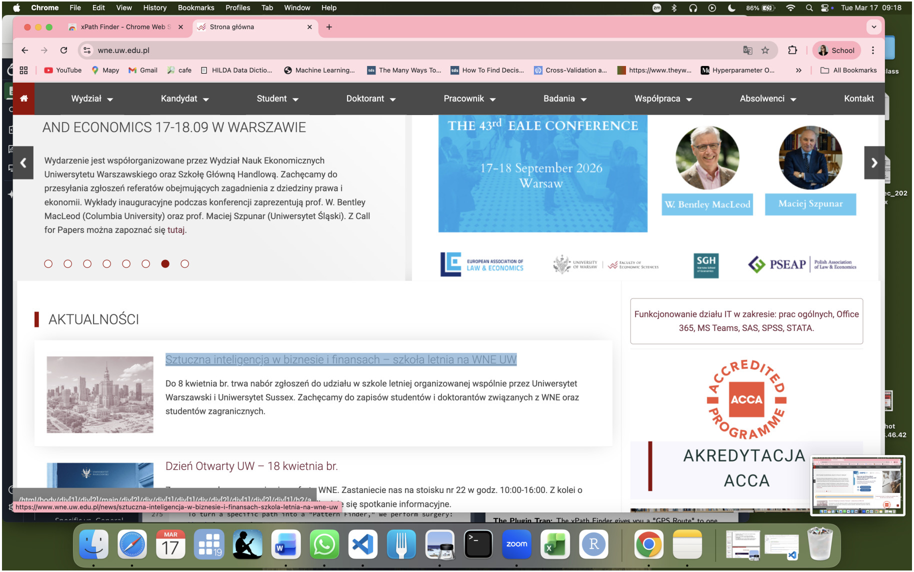
ChroPath plugin



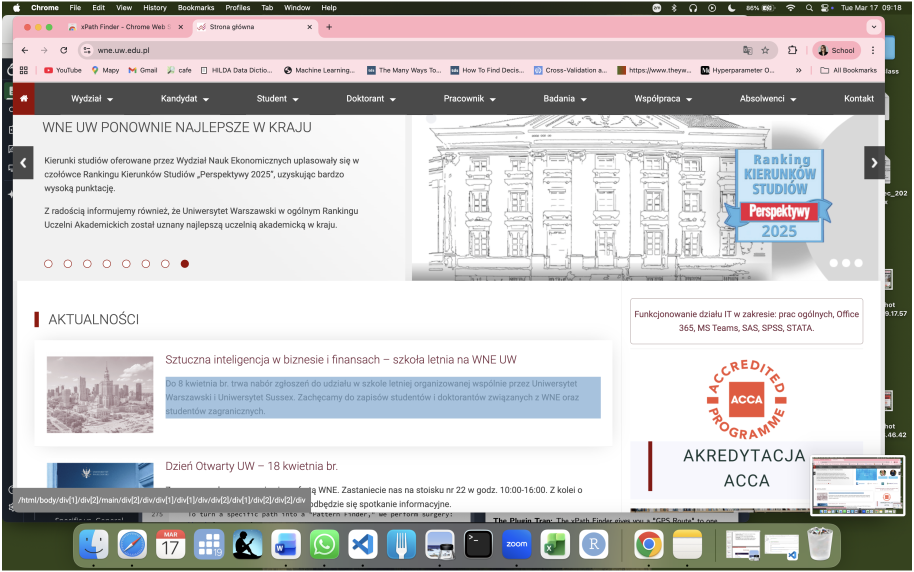
Example 1 - H1


wne.uw.edu.pl
Wydział
Kandydat
Student
Doktorant
Pracownik
Badania
Współpraca
Absolwenci
Kontakt
SPOTKANIE EUROPEAN HEALTH POLICY GROUP
Warsztaty poświęcone będą zagadnieniom związanym z reformami podstawowej i środowiskowej opieki zdrowotnej na przykładzie krajów Europy, Ameryki Północnej, Australii, Nowej Zelandii oraz Chin. Na konferencję zapraszamy w dniach 7-8 maja br.
Z wykładem specjalnym wystąpi Piotr Włodarczyk - dyrektor Departamentu Analiz i Hurtowni Danych Centrum e-Zdrowia. Rejestracji można dokonać za pośrednictwem formularza – zapraszamy do zapisów: https://opos.wne.uw.edu.pl/product/spring-meeting-2026/
EHPG
How can we bring about radical reform in primary and community care?
A workshop and special issue of HEPL to facilitate learning from diverse contexts and systems
Library of the University of Warsaw
Dobra 56/66, 00-312 Warsaw
7th - 8th May, 2026
AKTUALNOŚCI
Sztuczna inteligencja w biznesie i finansach – szkoła letnia na WNE UW
Do 8 kwietnia br. trwa nabór zgłoszeń do udziału w szkole letniej organizowanej wspólnie przez Uniwersytet Warszawski i Uniwersytet Sussex. Zachęcamy do zapisów studentów i doktorantów związanych z WNE oraz
Funkcjonowanie działu IT w zakresie: prac ogólnych, Office 365, MS Teams, SAS, SPSS, STATA.
ACCREDITED PROGRAMME
ACCA
/html/body/div[1]/div[2]/main/div[2]/div/div[1]/div[1]/div/div[1]/h1

Example 2 - H2


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Tue Mar 17 09:18
xPath Finder - Chrome Web S
Strona główna
wne.uw.edu.pl
School
YouTube
Mapy
Gmail
cafe
HILDA Data Dictio...
Machine Learning...
The Many Ways To...
How To Find Decis...
Cross-Validation a...
https://www.theyw...
Hyperparameter O...
All Bookmarks
Wydział
Kandydat
Student
Doktorant
Pracownik
Badania
Współpraca
Absolwenci
Kontakt
AND ECONOMICS 17-18.09 W WARSZAWIE
Wydarzenie jest współorganizowane przez Wydział Nauk Ekonomicznych Uniwersytetu Warszawskiego oraz Szkołę Główną Handlową. Zachęcamy do przesyłania zgłoszeń referatów obejmujących zagadnienia z dziedziny prawa i ekonomii. Wykłady inauguracyjne podczas konferencji zaprezentują prof. W. Bentley MacLeod (Columbia University) oraz prof. Maciej Szpunar (Uniwersytet Śląski). Z Call for Papers można zapoznać się tutaj.
THE 43rd EALE CONFERENCE
17–18 September 2026
Warsaw
W. Bentley MacLeod
Maciej Szpunar
EUROPEAN ASSOCIATION OF LAW & ECONOMICS
SGH
PSEAP
AKTUALNOŚCI
Sztuczna inteligencja w biznesie i finansach – szkoła letnia na WNE UW
Do 8 kwietnia br. trwa nabór zgłoszeń do udziału w szkole letniej organizowanej wspólnie przez Uniwersytet Warszawski i Uniwersytet Sussex. Zachęcamy do zapisów studentów i doktorantów związanych z WNE oraz studentów zagranicznych.
Dzień Otwarty UW – 18 kwietnia br.
WNE. Zastaniecie nas na stoisku nr 22 w godz. 10:00-16:00. Z kolei o
się spotkanie informacyjne.
/html/body/div[1]/div[2]/main/div[2]/div/div[1]/div[1]/div/div[2]/div[1]/div[2]/div[1]/h2/a
https://www.wne.uw.edu.pl/news/sztuczna-inteligencja-w-biznesie-i-finansach-szkola-letnia-na-wne-uw
Funkcjonowanie działu IT w zakresie: prac ogólnych, Office 365, MS Teams, SAS, SPSS, STATA.
ACCREDITED
ACCA
PROGRAMME
AKREDYTACJA
ACCA
MAR
17

Example 3 - div



Final Workflow: Plugin to Python

1. **Target Selection:** Open *xPath Finder* on the WNE website.
2. **Address Retrieval:** Click a news title to get the **Relative XPath**.
3. **Implementation:**
   - Use `requests` to download the page.
   - Use `lxml` to find the data using the copied path.
   - Add `/text()` to extract only the human-readable words.
4. **Output:** Save the results to a **.csv** file for data analysis.

Lesson: Even without an API, we can build a structured database by finding the "patterns" in the HTML code.

XPath Strategy: Long vs. Short

- **The "Fragile" Path (Absolute):**
  `/html/body/div[1]/div[2]/main/div[2]/div/div[1]/h2/a`
  - One tiny change to the website layout kills your scraper.
- **The "Robust" Path (Relative):**
  `//h2/a`
  - It means: "Find any `<h2>` that has a link inside, anywhere on the page."
  - Much harder to break!

Code Tip: Always add `/text()` to your XPath if you want the actual words and not the HTML brackets!

Specific vs. General

The Plugin Trap: The xPath Finder gives you a "GPS Route" to one specific box (e.g., `div[8]`).**The Project Goal:** We don't want the 8th box; we want **EVERY** box.

The Logic of the "Shave"

To turn a specific path into a "Pattern Finder," we perform surgery:

1. **Cut the Head:** Remove `/html/body/div...` and start with `//main`.
2. **Delete the ID:** Delete the numbers in brackets (like `[8]` or `[2]`).
3. **The Vacuum:** Add `//text()` to suck the words out of all found boxes.

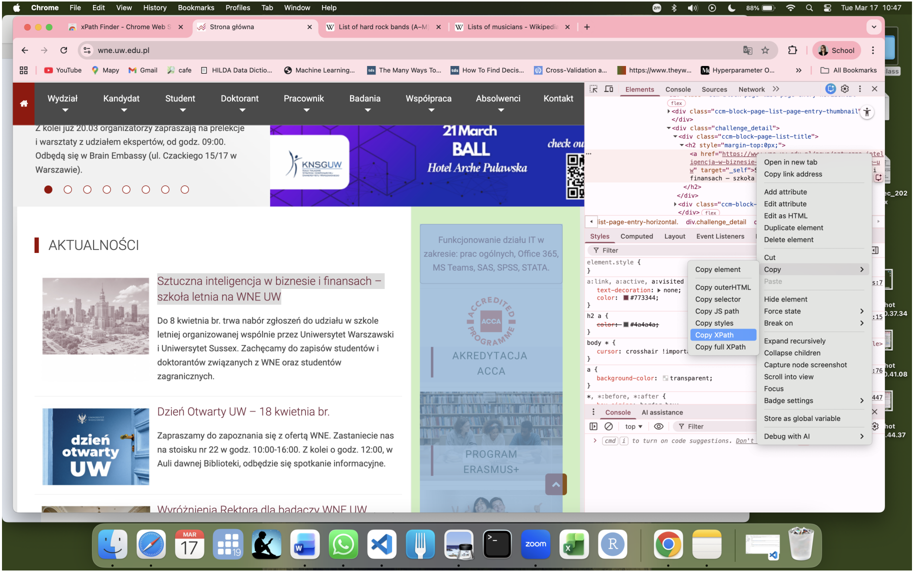
Example of the Transformation:

- **Stupid Path:** `/html/body/.../div[8]/div[2]/div[2]/div` → (Finds 1 thing)
- **Smart Path:** `//main//div/div/div/text()` → (Finds **EVERYTHING**)

Chrome Developer Tools: The Copy Menu

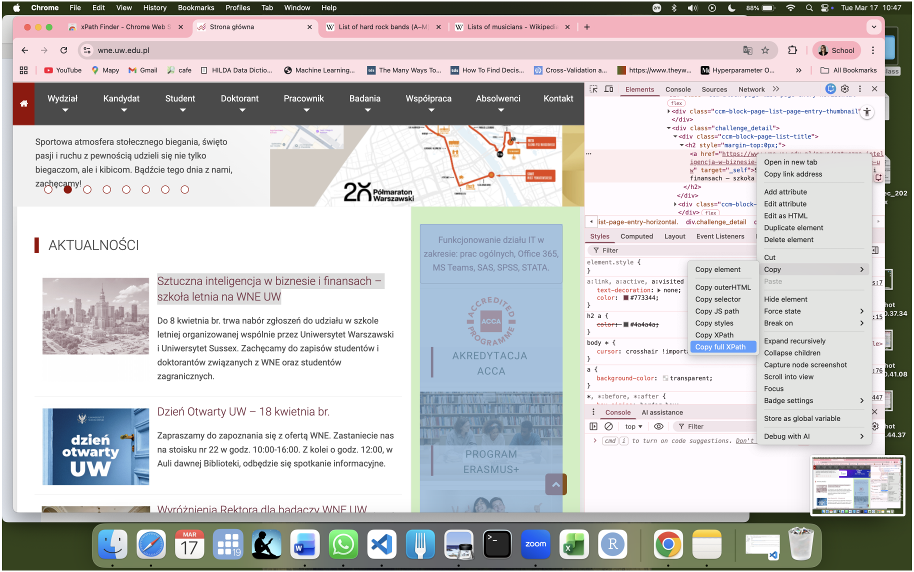
- **Copy XPath:** Returns a **Relative Path**. It looks for the nearest `id` attribute. If no `id` exists, it gives the shortest possible path. *(Use this one!)*
- **Copy full XPath:** Returns the **Absolute Path**. It traces every single tag from the `<html>` root. *(Avoid this for scraping!)*
- If you copy the path for one band on Wikipedia, it might look like: `/html/body/div[1]/div[2]/main/div[2]/div/div[1]/div[1]/div`
- To get **all** bands, change it to: `//*[@id="content"]/div[1]/div/div[2]/div[1]/div[2]/div[1]/`

Copy XPath:




```
//*[@id="content"]/div[1]/div/div[2]/div[1]/div[2]/div[1]/h2/a
```

Copy full XPath:



/html/body/div[1]/div[2]/main/div[2]/div/div[1]/div[1]/div/div[2]/div[1]/di

Static and Dynamic Websites

Definitions

- **Static:** Appears exactly as written. No changes after loading.
- **Server-side Dynamic:** Content generated by server scripts before reaching the browser.
- **Client-side Dynamic:** Content modified by scripts (e.g., JavaScript) running in the user's browser.

The Reality

Purely static websites are rare; most modern sites include at least small scripts.

Scraping: Static vs. Dynamic

Static Scraping

- Sends a direct request.
- Returns HTML/XML nodes.
- Requires XPath, CSS selectors, or RegEx.

Dynamic Scraping

- Simulates user interaction.
- Requires browser automation (clicking, scrolling).
- Executes custom scripts to trigger elements.

Scaling the Scraper

- **The First Step:** Successfully scrape a *single* page.
- **The Assumption:** Similar pages share a similar structure.
- **Acquiring Links:**
  1. **Generation:** If links follow a numerical pattern (e.g., `page/1`, `page/2`).
  2. **Extraction:** Scraping links from a directory or "hub" page.

Efficiency Tip

Before searching for unknown links, estimate the probability of success vs. the time cost.

Handling Exceptions and Crashes

- Scaling leads to **unexpected exceptions** (bad links, server timeouts).
- **Strategy:** Don't make the code too complex. Divide the project into smaller components.
- **Debugging:** Use `print()` statements to track progress.

Resource Protection

- Save data **periodically** to prevent total loss.
- Avoid storing everything in one massive DataFrame; use smaller chunks.
- Manage memory carefully.

Ensuring Scraper Efficiency

Performance Factors:

- **Time:** Slowed down by processing functions, excessive prints, and frequent saves.
- **Data Quality:** Target valuable output; avoid irrelevant data collection.

Storage in Python:

- Use `pandas` or `numpy`.
- Save as **CSV, TXT, or Pickle**.
- **Pro Tip:** Specify object size *a priori* (before starting) to avoid slow dynamic resizing.

References

- Mitchell, R. (2024). *Web Scraping with Python*. O'Reilly Media, Inc. Chapter 3.